HUMAIN HACKATHON · CHALLENGE 2: DELIVER AI WITH LESS COMPUTE

OptiRoute

A bilingual (Arabic / English) compute cascade that sends each task to the cheapest tier able to answer it correctly, with every number measured on recorded real model runs.

−79.6%
compute vs always-large-model (kTok × active params)

4.7×
cheaper per successful task (CPS)

86.1%
accuracy on 36 held-out cases (always-14B: 88.9%)

1.7 ms
total CPU overhead per task for all 4 guard components

Nawaf Alduaij

Runs anywhere from a laptop to a cloud API · 4 deployment profiles benchmarked · headline measured on one consumer GPU (RX 7900 XTX) · deterministic replay in 4 s

THE ISSUE

Most enterprise requests overpay for compute

The default setup sends every request to the largest model with the whole document attached.

Measured on our 36 held-out bilingual tasks:

- Always-14B on raw context: 192 kTok·B, 13,712 tokens, 88.9% accuracy
- Always-3B: 79.1% cheaper and only 2.8 pts less accurate, so most tasks never needed the large model
- Pruning alone removes 70.1% of context tokens; with 4× distractor noise, 94.0%
- A naive semantic cache returns the answer for #INV-8841 when asked about #INV-8842: a cheap but wrong result

WHAT THE CHALLENGE ASKS, AND WHERE OPTIROUTE ANSWERS IT

1 **Model choice per task**
route easy work to the 3B model

2 **Context that earns its tokens**
pruning + boilerplate removal

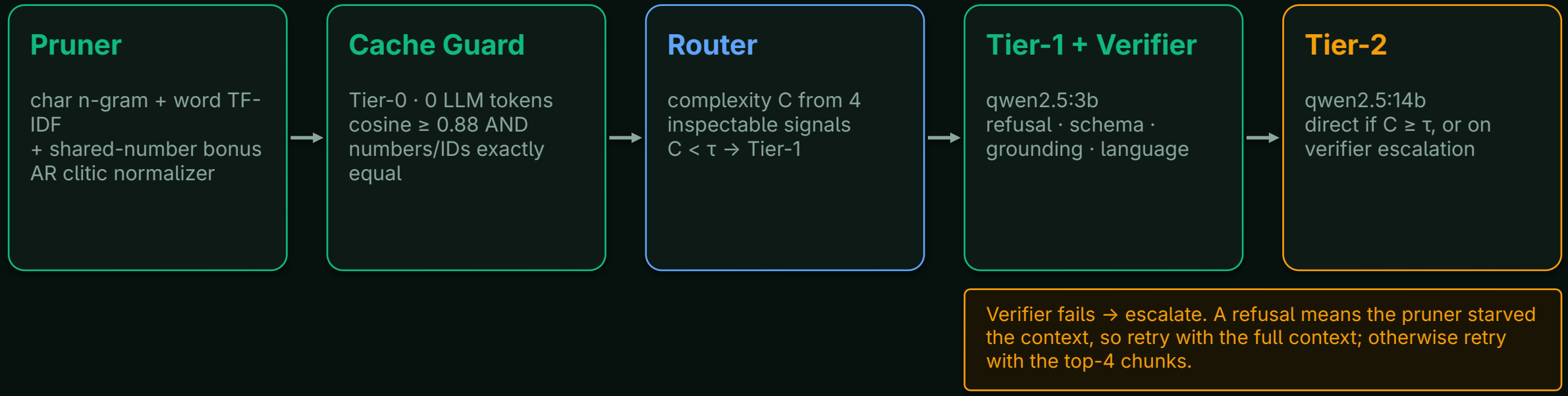
3 **Verification where it pays**
< 1 ms rule checks, escalate only on failure

4 **Cost per successful task**
CPS reported for every configuration

5 **Accuracy & latency floors**
graded by a separate ground-truth scorer

OUR SOLUTION

A 3-tier cascade with deterministic CPU guards between the tiers



Adaptation loop: a W = 20 window of Tier-1 verifier outcomes and P95 latency moves $\tau \in [0.25, 0.75]$. τ tightens by 0.08 when the pass rate drops below 70% or P95 exceeds the SLO, and recovers by 0.05 per step after 5 healthy steps (hysteresis).

Every component is deterministic, CPU-only and individually timed: mean pruner 1.36 ms · cache 0.18 ms · router 0.02 ms · verifier 0.12 ms.

HOW IT WORKS

Four steps, each one measurable and switchable for ablation

1 Context pruning & numeric-guarded cache

- Paragraphs scored: 0.50·cos(char 3–5g) + 0.35·cos(word TF-IDF) + 0.15·shared numbers/IDs
- Arabic: Alef/Teh-Marbuta/Alef-Maksura unified, tashkeel stripped, proclitics (وال، بال، لل...) removed
- Cache key = query wording (digits masked) + hash of top-2 chunks; hit only if entity sets are identical

2 Complexity routing & Tier-1 execution

- C = 0.30 multi-hop + 0.30 arithmetic + 0.25 strict JSON/table + 0.15 relevance dispersion
- C < τ → qwen2.5:3b on the pruned context; C ≥ τ → qwen2.5:14b directly
- Measured: 28 of 36 tasks finished on Tier-1, 2 on Tier-0 cache

3 Verification & full-context escalation

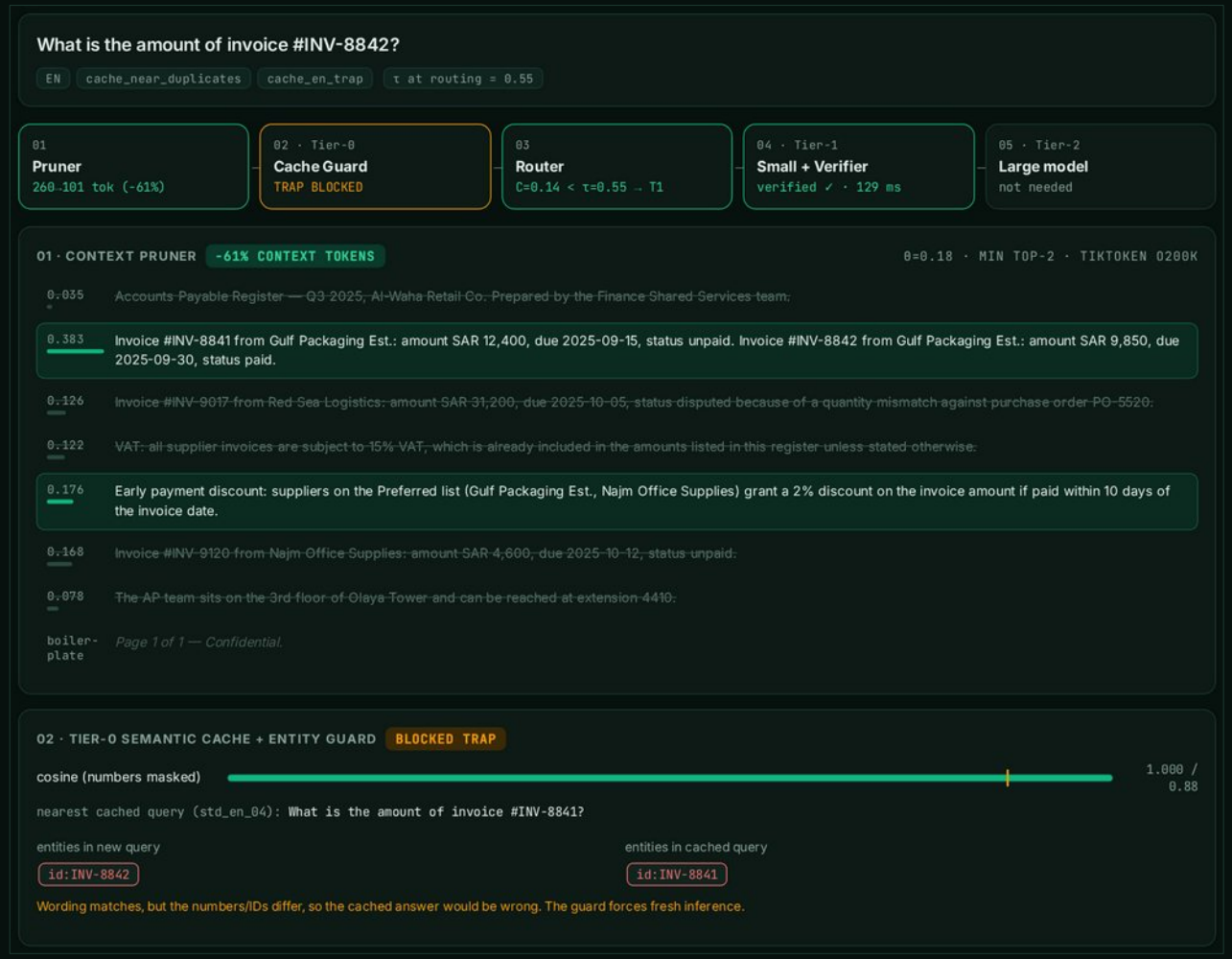
- Refusal/degeneracy · JSON schema · numeric grounding (each output number must appear in, or follow by + − × ÷ % from, the context) · Arabic-in/Arabic-out
- Precision 100% and recall 57% on recorded Tier-1 outputs; never sees ground truth
- Refusal → Tier-2 with full context (recovers pruner misses); other failures → top-4 relaxed context

4 Bidirectional τ adaptation + shadow audit

- Estimated Tier-1 health = verifier pass rate × agreement on sampled Tier-2 audits (every 4th verified answer)
- Tightens when health < 70% or P95 > SLO; recovers +0.05/step after 5 healthy steps (hysteresis)
- Recorded wave: τ 0.55 → 0.47 in the hard burst (request 26) → back to 0.55 (requests 38, 39)

IMPLEMENTATION & LIVE DEMO

A working prototype you can click through



Live UI: the #INV-8842 trap is blocked by the entity guard (wording cosine 1.000, IDs differ)

WHAT WAS BUILT

- Cascade engine (src/): pruner, entity-guarded cache, router, verifier, adaptive τ, orchestrator; every component can be switched off for ablation
- Interactive UI: 9 bilingual presets replay recorded runs; custom questions run on the live models; every stage visualised with its cost vs always-14B
- evaluate.py: --record / --replay / --sim, 2 ablation tables, 5 robustness experiments
- 35 unit tests; slides and reports generated from measured results

TECH STACK

Python · any OpenAI-compatible endpoint (Ollama, vLLM, Groq…) · 4 deployment profiles (profiles/*.env) · measured with qwen2.5/3.5, llama3.2, ALLaM-2, gpt-oss-120b · OpenAI-compatible client · tiktoken · FastAPI + Chart.js · pytest · python-pptx

github.com/nawafalduaij/optiroute

EVALUATION METHODOLOGY

Held-out, recorded once, replayed, graded separately

Category	Cases	What it stresses
standard	20	single-chunk policy / invoice lookups
low lexical overlap	4	paraphrased query, almost no shared keywords
multi hop	4	facts from 2 chunks + arithmetic (SLA credit, penalties)
arabic orthographic variants	4	missing hamza, ى/ي, ة/ه, clitics, mixed numerals
cache near duplicates	4	2 paraphrase hits + 2 traps (#INV-8841→8842, 6→8 days)
total	**36**	**21 English / 15 Arabic · separate 10-case dev set**

Record & replay

180 real calls (36 tasks × raw/pruned/relaxed × tier), temperature 0, models pre-warmed so load time is excluded, prompt hash stored to detect drift

No circular metrics

Accuracy comes from a ground-truth scorer (numeric tolerance, required facts, JSON fields). The runtime verifier is graded against it, never used for it.

Measured units

tiktoken o200k token counts · compute = Σ tokens/1000 × active params (3B / 14B, cache = 0) · latency = recorded LLM time + perf_counter CPU time

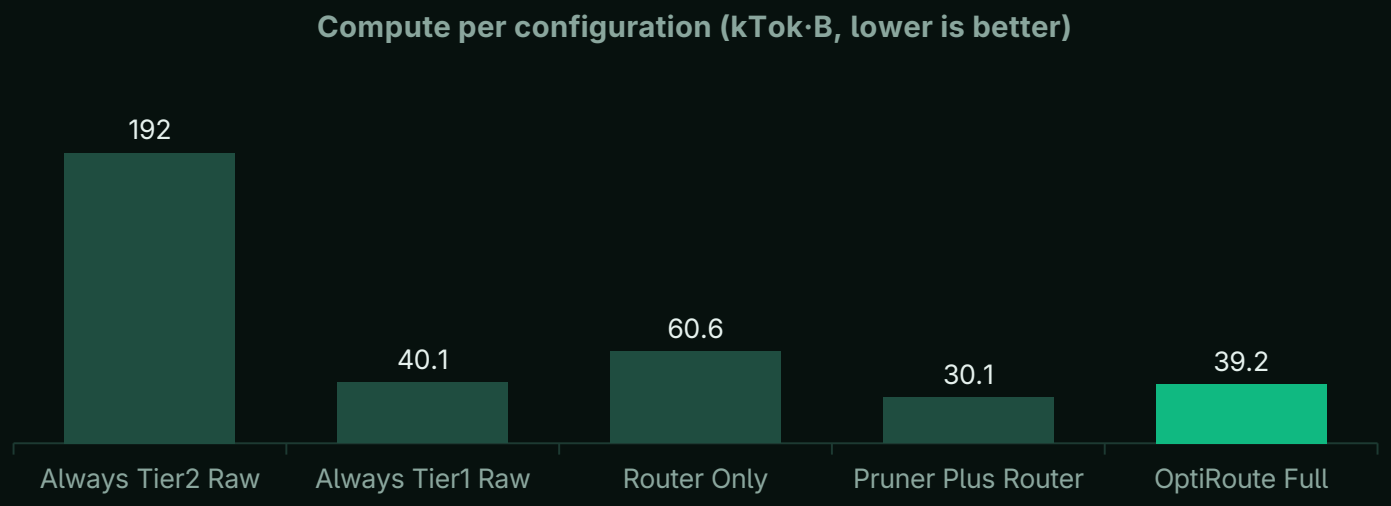
Reproducible

Full evaluation replays in 4.0 s without a GPU or API keys; 35 unit tests in sim mode

RESULTS: HELD-OUT, RECORDED REAL RUNS

Cumulative build-up: what each component adds

Configuration	Accuracy	Tokens	Compute kTok·B	Δ compute	CPS	P50 ms	Tier 0/1/2	False cache hits
1 Always Tier2 Raw	88.9% (32/36)	13,712	192.0	+0.0%	6.00	473	0/0/36	0/2
2 Always Tier1 Raw	86.1% (31/36)	13,363	40.1	-79.1%	1.29	129	0/36/0	0/2
3 Router Only	86.1% (31/36)	13,580	60.6	-68.4%	1.96	146	0/32/4	0/2
4 Pruner Plus Router	80.6% (29/36)	6,519	30.1	-84.3%	1.04	148	0/32/4	0/2
5 OptiRoute Full	**86.1% (31/36)**	**6,847**	**39.2**	**-79.6%**	**1.26**	**152**	**2/28/6**	**0/2**



Full cascade vs always-14B

- −79.6% compute, −50.1% tokens
- CPS 1.26 vs 6.00 kTok·B per success (4.7× cheaper)
- P50 latency 152 ms vs 473 ms
- Accuracy 86.1% vs 88.9%: 1 fewer correct answers out of 36

ABLATIONS & ROBUSTNESS

Leave-one-out, other models, and adaptation on real runs

Configuration	Accuracy	Tokens	Compute kTok·B	Δ compute	CPS	P50 ms	Tier 0/1/2	False cache hits
Full OptiRoute	**86.1% (31/36)**	**6,847**	**39.2**	**-79.6%**	**1.26**	**152**	**2/28/6**	**0/2**
LOO No Cache	86.1% (31/36)	7,255	40.4	-79.0%	1.30	152	0/30/6	0/2
LOO No Cache Entity Guard	83.3% (30/36)	6,401	35.1	-81.7%	1.17	148	4/27/5	2/2
LOO No Pruner	86.1% (31/36)	13,235	64.5	-66.4%	2.08	142	2/29/5	0/2
LOO No Verifier	80.6% (29/36)	6,111	28.9	-85.0%	1.00	143	2/30/4	0/2
LOO No Adaptation	86.1% (31/36)	6,847	39.2	-79.6%	1.26	152	2/28/6	0/2

DEPLOYMENT PROFILES: same cascade code, laptop to cloud (all recorded real runs)

Profile	Tier-1 → Tier-2	Tier-1 alone	Always T2	OptiRoute	Sent to T2	T2 tokens
single-gpu	**qwen2.5:3b → qwen2.5:14b**	**86.1%**	**88.9%**	**86.1%**	**6/36**	**-88%**
hybrid	qwen3.5:9b → gpt-oss-120b	91.7%	97.2%	100.0%	7/36	-84%
edge	qwen2.5:0.5b → gpt-oss-120b	55.6%	97.2%	83.3%	8/36	-83%
cloud	allam-2-7b → gpt-oss-120b	94.4%	97.2%	86.1%	6/36	-87%

Entity guard 0/2 vs 2/2 false hits · verifier +5.6 pts · pruner −48% tokens (leave-one-out, single-gpu)

Cloud profile: the cascade scores below ALLaM alone: refusal rules written for Qwen miss ALLaM's phrasing (see limitations).

REAL τ(t), WAVE WORKLOAD



Tightens in the burst, then recovers · +42% compute

HONEST LIMITATIONS & ROADMAP

What this prototype does not solve yet

1. Grounded-but-wrong arithmetic slips past the verifier

Recall is 57%: rule checks cannot tell 10% of the fee from 25% when both numbers are in the context. Multi-hop accuracy is 25% for the cascade, and even the 14B gets only 50% (it wrote 900×4 + 1200×3 = 4,500, and switched to Chinese mid-answer in Arabic).

2. A lexical pruner is blind to zero-overlap paraphrases

Low-overlap accuracy is 75% with the pruner vs 100% without it. Recovery only works when the 3B admits it doesn't know, and the refusal rules are model-specific: written for Qwen, they miss ALLaM's phrasing.

3. Adaptation needs a costly signal; small benchmark

Rule checks miss confident errors: verifier-only health never fell below 80%, so τ stayed flat. Shadow audits make τ react on real runs but cost +42% compute, and since complexity scores cluster, a 0.08 τ shift barely re-routes tasks. n = 36 synthetic docs: ±1 task = 2.8 pts.

PRODUCTION ROADMAP

- Calculator tool + program-of-thought for multi-hop math, with the verifier recomputing the expression
- Per-model verifier calibration on the dev set (refusal phrasing differs between model families)
- Small bilingual embedding re-ranker as a fallback when the lexical scores are flat
- Token-logprob confidence as a 5th verifier signal for confident-but-wrong answers
- Output language guard on Tier-2 (catches code-switching)
- Risk-weighted audit sampling + a larger dev set to calibrate τ, the SLO and the verifier
- Serve with vLLM + batching; swap in HUMAIN/ALLaM models through the same OpenAI-compatible client

TAKEAWAYS

Cheap only counts if it is right, and we measured both

−79.6%
compute vs always-14B

4.7×
cheaper per successful task

0/2
false cache hits with the entity guard

100%
verifier precision (no wasted escalations)

Why this design

- Evaluation first: a held-out set, recorded real runs, and leave-one-out ablations, instead of a prompt wrapper with hand-picked numbers
- Bilingual by construction: Arabic normalization in the pruner, cache and verifier
- Runs fully local on one 24 GB GPU: no rate limits, no API cost, data stays on-premise
- Hardware-agnostic: 4 deployment profiles from laptop to cloud, same code; 8 recorded model pairs
- Every guard is inspectable and costs about 1.7 ms of CPU time

github.com/nawafalduaij/optiroute

```
pip install -r requirements.txt
python evaluate.py --replay data/recorded_runs.jsonl
uvicorn app:app --port 8000   # live demo UI

Re-record on your own hardware:
python evaluate.py --record --fresh
```

Nawaf Alduaij · thank you